

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «КиСБ»
Тема: «Генерирование равномерно распределенных псевдослучайных
последовательностей»

Выполнил:
Студент 4-го курса
Группы АС-66
Горобец А.В.
Проверил:
Хацкевич А.С.

Брест 2026

Вариант 3

Цель работы: освоить основные алгоритмы программного генерирования равномерно распределенных псевдослучайных последовательностей.

Задание:

Часть 1:

1. Согласно варианту реализовать приложение, генерирующие псевдослучайную равномерно распределенную последовательность произвольной длины из заданного алфавита.

2. Подобрать параметры данного генератора таким образом, чтобы период последовательности имел максимальное значение.

3. Используя статистическое тестирование, проверить гипотезу о том, что сгенерированная последовательность действительно имеет равномерное распределение (см. приложение 2).

4. Отчет должен обязательно содержать несколько тестов, отображающих изменение статистических свойств последовательностей в зависимости от параметров генератора.

3	Генератор Эйхенауэра – Лена с обращением	N=20	
---	--	------	--

```
import java.util.*;

public class lab1p1 {

    public static int modInverse(int val, int n) {
        int a = val;
        int b = Math.abs(n);
        int u1 = 1;
        int u2 = 0;
        while (a > 0) {
            int q = b / a; // частное
            int remainder = b % a; // остаток
            int uNext = u2 - q * u1;
            b = a;
            a = remainder;
            u2 = u1;
            u1 = uNext;
        }
        if (b > 1) {
            return 0;
        }
        if (u2 < 0) {
            u2 += Math.abs(n);
        }
        return u2;
    }

    public static ArrayList<Integer> generator(int paramA, int paramC, int paramN, int x0, int sLength) {
        int currentX = x0;
        ArrayList<Integer> subsequence = new ArrayList<>();
        for (int i = 0; i < sLength; i++) {
            subsequence.add(currentX);
            if (currentX == 0) {
                currentX = paramC;
            }
        }
    }
}
```

```

    } else {
        int invertX = modInverse(currentX, paramN);

        if (invertX == 0) {
            currentX = paramC % paramN;
        } else {
            currentX = (paramA * invertX + paramC) % paramN;
        }
    }
}
return subsequence;
}

public static void periodTest(ArrayList<Integer> subsequence) {
    System.out.println("Последовательность: " + subsequence);
    int x0 = subsequence.get(0);
    boolean found = false;
    for (int i = 1; i < subsequence.size(); i++) {
        if (x0 == subsequence.get(i)) {
            System.out.println("Период = " + i);
            found = true;
            break;
        }
    }
    if (!found) {
        System.out.println("Период не обнаружен");
    }
}

public static void frequencyTest(ArrayList<Integer> subsequence) {
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < subsequence.size(); i++) {
        int num = subsequence.get(i);
        map.put(num, map.getOrDefault(num, 0) + 1);
    }
    System.out.println("Частоты: " + map);

    // Среднее
    double sum = 0;
    for (int num : subsequence) {
        sum += num;
    }
    double avg = sum / subsequence.size();
    System.out.println("Среднее: " + avg);

    // Дисперсия
    double varianceSum = 0;
    for (int num : subsequence) {
        varianceSum += Math.pow(num - avg, 2);
    }
    double variance = varianceSum / subsequence.size();
    System.out.println("Дисперсия: " + variance);
}

/**
 * Метод проверки равномерности распределения по критерию Пирсона (Хи-квадрат)
 * @param subsequence сгенерированная последовательность чисел
 * @param paramN модуль N (верхняя граница диапазона генератора)
 * @param numIntervals количество интервалов разбиения (например, 8)
 */
public static void checkChiSquare(ArrayList<Integer> subsequence, int paramN, int numIntervals) {
    System.out.println("\n=== КРИТЕРИЙ ХИ-КВАДРАТ (Проверка на равномерность) ===");
    int totalElements = subsequence.size();

```

```

int[] observed = new int[numIntervals];
double intervalWidth = (double) paramN / numIntervals;

// 1. Подсчет количества попаданий в каждый интервал
for (int num : subsequence) {
    int index = (int) (num / intervalWidth);
    if (index >= numIntervals) {
        index = numIntervals - 1; // Защита от вылета за границы массива
    }
    observed[index]++;
}

// 2. Вычисление теоретической частоты попадания
double expected = (double) totalElements / numIntervals;

// 3. Вычисление эмпирического значения хи-квадрат
double chiSquare = 0.0;
System.out.println("Распределение элементов по интервалам:");
for (int i = 0; i < numIntervals; i++) {
    double diff = observed[i] - expected;
    chiSquare += (diff * diff) / expected;
    System.out.printf(" Интервал %d [%3.0f - %3.0f]: получено = %3d, ожидалось = %.1f\n",
        i + 1, i * intervalWidth, (i + 1) * intervalWidth - 1, observed[i], expected);
}

System.out.printf("\nНаблюдаемое значение Chi-Square (хи-квадрат): %.4f\n", chiSquare);

// 4. Сравнение с критическим значением для alpha = 0.05 и df = numIntervals - 1
if (numIntervals == 8) {
    double criticalValue = 14.07; // Критическое значение для df = 7 при alpha = 0.05
    System.out.println("Критическое значение Chi-Square (alpha = 0.05, df = 7): " + criticalValue);

    if (chiSquare < criticalValue) {
        System.out.println("РЕЗУЛЬТАТ: Гипотеза о равномерности ПОДТВЕРЖДАЕТСЯ (хи-квадрат < "
            + criticalValue + ")");
    } else {
        System.out.println("РЕЗУЛЬТАТ: Гипотеза о равномерности ОТКЛОНЯЕТСЯ (хи-квадрат >= "
            + criticalValue + ")");
    }
}

public static void main(String[] args) {
    int paramN = 256;
    ArrayList<Integer> subsequence = generator(9, 14, paramN, 1, 100);

    periodTest(subsequence);
    frequencyTest(subsequence);

    // Запуск проверки на равномерность (8 интервалов)
    checkChiSquare(subsequence, paramN, 8);
}
}

```

```
C:\Program Files\Java\jdk-20\bin>java.exe -Djavagent=IntelliJ IDEA Community Edition 2023.2 \I\lib\idea_r.jar@51926% -Dfile.encoding=UTF-8 --daemon.stdout.encoding=UTF-8 --daemon.stderr.encoding=UTF-8 -classpath C:\Users\A... Последовательности: [1, 23, 237, 27, 185, 223, 37, 35, 241, 167, 221, 43, 169, 111, 21, 51, 225, 55, 205, 59, 153, 255, 5, 67, 209, 199, 189, 75, 137, 145, 245, 83, 193, 87, 173, 91, 121, 31, 229, 99, 177, 231, 157, 107, 1 ... Период не обнаружен Частоты: {1=1, 5=1, 7=1, 9=1, 15=1, 21=1, 23=1, 25=1, 27=1, 31=1, 35=1, 37=1, 39=1, 41=1, 43=1, 45=1, 51=1, 55=1, 57=1, 59=1, 61=1, 63=1, 65=1, 67=1, 71=1, 73=1, 75=1, 77=1, 81=1, 83=1, 87=1, 89=1, 91=1, 93=1, 95=1, 97=1, Среднее: 128.32 Дисперсия: 6626.337599999999 === КРИТЕРИЙ ХИ-КВАДРАТ (Проверка на равенство) === Распределение элементов по интервалам: Интервал 1 [ 0 - 31]: получено = 10, ожидалось = 12,5 Интервал 2 [ 32 - 63]: получено = 12, ожидалось = 12,5 Интервал 3 [ 64 - 95]: получено = 13, ожидалось = 12,5 Интервал 4 [ 96 - 127]: получено = 14, ожидалось = 12,5 Интервал 5 [ 128 - 159]: получено = 14, ожидалось = 12,5 Интервал 6 [ 160 - 191]: получено = 15, ожидалось = 12,5 Интервал 7 [ 192 - 223]: получено = 16, ожидалось = 12,5 Интервал 8 [ 224 - 255]: получено = 8, ожидалось = 12,5 Наблюдаемое значение Chi-Square (хи-квадрат): 3,2000 Критическое значение Chi-Square ( $\alpha$ =0.05, df = 7): 14.07 РЕЗУЛЬТАТ: Гипотеза о равномерности ПОТВЕРЖДАЕТСЯ (хи-квадрат < 14.07)
```

Часть 2:

1. Реализовать приложение, позволяющее генерировать простое число по следующей схеме (с использованием, например, теста Рабина - Миллера)
2. В отчете обязательно должны присутствовать тесты, подтверждающие правильность разработанных программ.

3	Полиномиальный тест распознавания простоты.
---	---

```
import java.math.BigInteger;
```

```
public class lab1p2gen2 {
```

```
// 1. Генератор Эйхенауэра-Лена с обращением (модуль N = 2^16)
```

```
public static int modInverse(int val, int n) {
```

```
int a = val, b = n;
```

```
int u1 = 1, u2 = 0;
```

```
while (a > 0) {
```

```
int q = b / a;
```

```
int remainder = b % a;
```

```
int uNext = u2 - q * u1;
```

```
b = a; a = remainder;
```

```
u2 = u1; u1 = uNext;
```

$$\}$$

```
if (b > 1) return 0;
```

```
return (u2 < 0) ? u2 + n : u2;
```

$$\}$$

```
public static int nextEichenauerLehn(int currentX, int paramA, int paramC, int paramN) {
```

```
if (currentX == 0) {
```

```
return paramC % paramN;
```

$$\}$$

```
int inv = modInverse(currentX, paramN);
```

```
if (inv == 0) {
```

```
return (paramA * (currentX + 1) + paramC) % paramN;
```

$$\}$$

```
return (paramA * inv + paramC) % paramN;
```

$$\}$$

```
// Генерация р-битного кандидата
```

```
public static BigInteger generateCandidate(int bits, int paramA, int paramC, int paramN, int[] stateX) {
```

```
BigInteger number = BigInteger.ZERO;
```

```
for (int i = 0; i < bits; i++) {
```

```
stateX[0] = nextEichenauerLehn(stateX[0], paramA, paramC, paramN);
```

```
int bit = (stateX[0] > (paramN / 2)) ? 1 : 0;
```

```
if (bit == 1) {
```

```
number = number.setBit(i);
```

```

    }
}
// Установка старшего и младшего битов в 1
number = number.setBit(bits - 1);
number = number.setBit(0);
return number;
}

// 2. Список малых простых чисел для сита (< 256)
private static final int[] PRIMES = {
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
    73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151,
    157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233,
    239, 241, 251
};

/**
 * Фильтрация ситом малых простых чисел.
 * Возвращает делитель, если число составное, или -1, если прошло сито.
 */
public static int getSieveDivisor(BigInteger n) {
    for (int p : PRIMES) {
        BigInteger num = BigInteger.valueOf(p);
        if (n.equals(num)) return -1; // Число само является простым
        if (n.mod(num).equals(BigInteger.ZERO)) {
            return p; // Нашли малый делитель
        }
    }
    return -1; // Прошло сито
}

// 3. Полиномиальный тест простоты (AKS / Теорема 3)
public static BigInteger[] multiplyPoly(BigInteger[] poly1, BigInteger[] poly2, int r, BigInteger n) {
    BigInteger[] result = new BigInteger[r];
    for (int i = 0; i < r; i++) result[i] = BigInteger.ZERO;

    for (int i = 0; i < r; i++) {
        if (poly1[i].equals(BigInteger.ZERO)) continue;
        for (int j = 0; j < r; j++) {
            if (poly2[j].equals(BigInteger.ZERO)) continue;

            int targetIndex = (i + j) % r;
            BigInteger term = poly1[i].multiply(poly2[j]).mod(n);
            result[targetIndex] = result[targetIndex].add(term).mod(n);
        }
    }
    return result;
}

public static BigInteger[] polyPow(BigInteger[] base, BigInteger exponent, int r, BigInteger n) {
    BigInteger[] result = new BigInteger[r];
    for (int i = 0; i < r; i++) result[i] = BigInteger.ZERO;
    result[0] = BigInteger.ONE;

    BigInteger[] curBase = base;
    BigInteger exp = exponent;

    while (exp.compareTo(BigInteger.ZERO) > 0) {
        if (exp.testBit(0)) {
            result = multiplyPoly(result, curBase, r, n);
        }
        curBase = multiplyPoly(curBase, curBase, r, n);
        exp = exp.shiftRight(1);
    }
}

```

```

    }
    return result;
}

public static boolean isPrimeSimple(BigInteger n) {
    if (n.compareTo(BigInteger.TWO) < 0) return false;
    BigInteger limit = n.sqrt();
    for (BigInteger d = BigInteger.TWO; d.compareTo(limit) <= 0; d = d.add(BigInteger.ONE)) {
        if (n.mod(d).equals(BigInteger.ZERO)) return false;
    }
    return true;
}

public static BigInteger getLargestPrimeFactor(BigInteger num) {
    BigInteger maxPrime = BigInteger.valueOf(-1);
    BigInteger temp = num;
    BigInteger two = BigInteger.valueOf(2);

    while (temp.mod(two).equals(BigInteger.ZERO)) {
        maxPrime = two;
        temp = temp.divide(two);
    }

    BigInteger i = BigInteger.valueOf(3);
    while (i.multiply(i).compareTo(temp) <= 0) {
        while (temp.mod(i).equals(BigInteger.ZERO)) {
            maxPrime = i;
            temp = temp.divide(i);
        }
        i = i.add(two);
    }
    if (temp.compareTo(two) > 0) maxPrime = temp;
    return maxPrime;
}

public static boolean aksTest(BigInteger n) {
    if (n.compareTo(BigInteger.ONE) <= 0) return false;

    // Шаг 0: Пробные деления
    for (int p : PRIMES) {
        BigInteger num = BigInteger.valueOf(p);
        if (n.equals(num)) return true;
        if (n.mod(num).equals(BigInteger.ZERO)) return false;
    }

    // Шаг 1: Проверка на совершенную степень (n = a^b)
    int maxB = n.bitLength();
    for (int b = 2; b <= maxB; b++) {
        BigInteger low = BigInteger.valueOf(2), high = n;
        while (low.compareTo(high) <= 0) {
            BigInteger mid = low.add(high).shiftRight(1);
            BigInteger res = mid.pow(b);
            int cmp = res.compareTo(n);
            if (cmp == 0) return false;
            if (cmp < 0) low = mid.add(BigInteger.ONE);
            else high = mid.subtract(BigInteger.ONE);
        }
    }

    // Шаги 2–7: Поиск r
    BigInteger r = BigInteger.valueOf(2);
    double log2n = n.bitLength();

```

```

while (r.compareTo(n) < 0) {
    if (!n.gcd(r).equals(BigInteger.ONE)) return false;

    if (isPrimeSimple(r)) {
        BigInteger rMinus1 = r.subtract(BigInteger.ONE);
        BigInteger q = getLargestPrimeFactor(rMinus1);

        double threshold = 4 * Math.sqrt(r.doubleValue()) * log2n;
        if (q.doubleValue() > threshold) {
            BigInteger exp = rMinus1.divide(q);
            if (!n.modPow(exp, r).equals(BigInteger.ONE)) {
                break;
            }
        }
    }
    r = r.add(BigInteger.ONE);
}

if (r.equals(n)) return true;

// Шаги 9–10: Полиномиальная проверка
int rInt = r.intValue();
long limit = (long) (2 * Math.sqrt(rInt) * log2n);

for (int a = 1; a <= limit; a++) {
    BigInteger bigA = BigInteger.valueOf(a);

    BigInteger[] basePoly = new BigInteger[rInt];
    for (int i = 0; i < rInt; i++) basePoly[i] = BigInteger.ZERO;
    basePoly[1] = BigInteger.ONE;
    basePoly[0] = n.subtract(bigA).mod(n);

    BigInteger[] leftPoly = polyPow(basePoly, n, rInt, n);

    BigInteger[] rightPoly = new BigInteger[rInt];
    for (int i = 0; i < rInt; i++) rightPoly[i] = BigInteger.ZERO;
    rightPoly[0] = n.subtract(bigA).mod(n);
    int nModR = n.mod(r).intValue();
    rightPoly[nModR] = rightPoly[nModR].add(BigInteger.ONE).mod(n);

    for (int i = 0; i < rInt; i++) {
        if (!leftPoly[i].equals(rightPoly[i])) return false;
    }
}

return true;
}

public static void main(String[] args) {
    int bits = 12; // Для AKS битовость 12-16 оптимальна
    int paramA = 9;
    int paramC = 14;
    int paramN = 65536;
    int[] stateX = {(int) (System.currentTimeMillis() % paramN)};
    if (stateX[0] % 2 == 0) stateX[0]++;

    System.out.println("=== ПОИСК ПРОСТОГО ЧИСЛА (Полиномиальный тест AKS) ===");
    BigInteger candidate = generateCandidate(bits, paramA, paramC, paramN, stateX);
    System.out.println("Сгенерированный начальный кандидат: " + candidate + "\n");

    int rejectedBySieve = 0;
    int rejectedByAKS = 0;
    long startTime = System.currentTimeMillis();

```



```

while (true) {
    int sieveDivisor = getSieveDivisor(candidate);

    if (sieveDivisor != -1) {
        // Число отсеяно ситом
        System.out.println("[СИТО] Отсеян кандидат: " + candidate + " (делится на " + sieveDivisor +
        ");");
        rejectedBySieve++;
        candidate = candidate.add(BigInteger.TWO);
        continue;
    }

    // Кандидат прошел предварительное сито
    System.out.println("\n[-->] Кандидат " + candidate + " прошел сито, запускаем тест AKS...");

    if (aksTest(candidate)) {
        long endTime = System.currentTimeMillis();
        System.out.println("\n=====");
        System.out.println("УСПЕХ! Найдено простое число: " + candidate);
        System.out.println("=====");
        System.out.println("Отсеяно ситом    : " + rejectedBySieve);
        System.out.println("Отсеяно тестом AKS: " + rejectedByAKS);
        System.out.println("Время поиска    : " + (endTime - startTime) + " ms");
        System.out.println("=====");
        break;
    } else {
        System.out.println("[AKS] Кандидат " + candidate + " отсеян тестом AKS (составное)\n");
        rejectedByAKS++;
        candidate = candidate.add(BigInteger.TWO);
    }
}
}
}

```

Вывод: в ходе работы освоены алгоритмы генерации равномерно распределенных псевдослучайных последовательностей на примере инверсного генератора Эйхенауэра — Лена. Практически подтверждена их равномерность с помощью статистических тестов и вычисления выборочной дисперсии. Изученные принципы успешно применены для формирования кандидатов в алгоритме поиска простых чисел.